



**UPO** UNIVERSITÀ DEL PIEMONTE ORIENTALE

Università Del Piemonte Orientale  
"Amedeo Avogadro"

Polo Formativo di Vercelli

Corso di laurea "Informatica" a.a 2024/2025

Progetto **MobiShare** per il corso di:

PROGETTAZIONE E IMPLEMENTAZIONE DI SISTEMI SOFTWARE IN RETE (MF0244)

Integrazione del modulo

APPLICAZIONI INTELLIGENTI (MF0781)

*Gruppo*

Baselice Asia (20049304)  
Duratti Matteo (20050673)  
Papa Marco Yuri (20051241)

# Indice

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Indice.....</b>                                                          | <b>2</b>  |
| <b>1. Specifica.....</b>                                                    | <b>3</b>  |
| 1.1 Scopo e contesto.....                                                   | 3         |
| 1.2 Requisiti Funzionali AI.....                                            | 3         |
| 1.3 Requisiti non funzionali.....                                           | 3         |
| 1.4 Scelte tecnologiche.....                                                | 4         |
| Architettura generale.....                                                  | 4         |
| Tech Stack.....                                                             | 4         |
| 1.5 Dominio.....                                                            | 4         |
| <b>2. Progettazione.....</b>                                                | <b>5</b>  |
| 2.1 Architettura generale.....                                              | 5         |
| 2.2 Architettura MVC.....                                                   | 5         |
| Model.....                                                                  | 5         |
| View.....                                                                   | 6         |
| Controller.....                                                             | 6         |
| 2.3 Rappresentazione Architettura.....                                      | 7         |
| Generale (con integrazione AI).....                                         | 7         |
| Specifica del Backend AI.....                                               | 7         |
| 2.4 Componenti principali.....                                              | 8         |
| 2.5 Struttura del database (Firestore).....                                 | 8         |
| Motivazioni della scelta di un database NoSQL (Specialmente Firestore)..... | 8         |
| Struttura dei Documenti.....                                                | 8         |
| Documento: face_encoding.....                                               | 8         |
| Documento: User.....                                                        | 9         |
| Documento: ml_models.....                                                   | 9         |
| Documento: ml_cache.....                                                    | 9         |
| Documento: agent_memory.....                                                | 9         |
| Documento: agent_session.....                                               | 10        |
| <b>3 Funzionalità AI.....</b>                                               | <b>10</b> |
| 3.1 FaceRecognition (Embedding e augmentation).....                         | 10        |
| 3.2 RandomForest (ML-Supervisionato).....                                   | 11        |
| Analisi Prestazioni.....                                                    | 12        |
| Fase 1: Dati Limitati (10 Giorni).....                                      | 13        |
| Fase 2: Aumento dei Dati (90 Giorni).....                                   | 13        |
| Fase 3: Aumento delle Feature e Dati a 6 Mesi.....                          | 14        |
| 3.3 Agente.....                                                             | 14        |
| Architettura e tecnologie usate.....                                        | 14        |
| Struttura Agente.....                                                       | 15        |
| Tools.....                                                                  | 15        |
| Definizione Identità.....                                                   | 17        |
| Processo di Reasoning.....                                                  | 17        |

<<NB: Questa relazione é un integrazione per il “modulo” di Applicazioni Intelligenti. qui verranno indicate le modifiche (aggiunte o rimozioni) rispetto al progetto di pissir .>>

# 1. Specifica

## 1.1 Scopo e contesto

Lo scopo del progetto MobiShare è la gestione di un servizio di mobilità condivisa. Oltre alle funzionalità di base (gestione utenti, noleggi, pagamenti), questo modulo introduce tre componenti del corso di Applicazioni Intelligenti, questi vengono usati per migliorare l'esperienza utente (sia cliente che gestore).

## 1.2 Requisiti Funzionali AI

**40. Verifica Facciale:** L'utente deve poter registrare il proprio volto dal profilo. Il sistema deve permettere la verifica biometrica (riconoscimento facciale) come alternativa alla password per autorizzare operazioni sensibili, come i pagamenti.

**41. Agente (Chatbot):** Il sistema fornisce un Agente chatbot. L'agente deve rispondere a domande e eseguire azioni (tramite *tools*) in base al ruolo dell'utente (Utente o Gestore).

**42. Analisi Predittiva (Machine Learning):** Il sistema deve analizzare i dati storici delle corse per addestrare un modello di machine learning. Il modello deve prevedere la domanda dei mezzi e fornire suggerimenti al gestore per ottimizzare la distribuzione della flotta.

## 1.3 Requisiti non funzionali

Per garantire l'efficacia e l'integrazione del modulo di Intelligenza Artificiale con il sistema MobiShare principale, sono stati definiti i seguenti requisiti non funzionali.

- **Gestione delle Dipendenze:** L'AI-Bridge dipende dal server MobiShare (Node.js). L'avvio del server AI deve fallire se il server principale non è raggiungibile (soddisfatto dalla funzione `ping_services` nel `lifespan` di FastAPI).
- **Isolamento dell'Architettura:** I servizi AI (FastAPI) non devono essere esposti pubblicamente. Tutta l'autenticazione e l'autorizzazione sono gestite dal backend Node.js.
- **Controllo degli Accessi (Role-Based):** L'agente applicare una rigorosa separazione dei permessi. Le funzionalità amministrative devono essere accessibili solo se il ruolo è gestore.
- **Privacy dei Dati Utente:** L'agente è istruito a non memorizzare informazioni sensibili (come dati di pagamento) nella sua memoria a lungo termine.
- **Robustezza FaceRecognition:** Il servizio Face Recognition deve gestire input non validi. Deve rifiutare le immagini che non contengono volti o che ne contengono più di uno e deve accertarsi di dare il permesso al legittimo proprietario.
- **Separazione dei Componenti:** La logica AI deve essere separata dalla logica principale (Node.js). Questo permette di aggiornare, riavviare e modificare i modelli ML e l'LLM in modo indipendente.

## 1.4 Scelte tecnologiche

### Architettura generale

- Architettura strutturata a servizi, dove ogni modulo AI é gestito da un servizio dedicato (`fr_service`, `rf_service`, `agent_service`).
- Accesso al database consentito solo tramite il backend.

### Tech Stack

- **Backend:**
  - Python 3.9+
  - Cmake
- **Librerie Principali:**
  - FastApi
  - Agno e AgentOS (Per l'agent)
  - Pandas e Numpy
  - sklearn (per il modello ML)
  - Matplot e Seaborn (per i plot)
  - OpenCV-2 (per l'augmentation delle immagini)
  - FaceRecognition (per il riconoscimento biometrico)
  - (vedi requirements.txt)
- **Database:**
  - Google Firebase.
- **API e Sistemi Esterni:**
  - Gemini 2.5 Flash (usato per l'agente e il reasoning).

## 1.5 Dominio

Il dominio rimane ugualmente articolato in due aree principali:

- **Area Utente:** Comprende il sistema di riconoscimento facciale usato come metodo di verifica per i pagamenti. Include anche un agente "chatbot" progettato per il supporto e permette all'utente di richiedere informazioni sul proprio account (es. `check_user_status` o `get_user_rides`) ed eseguire azioni come il riscatto dei punti (`redeem_shop_package`).
- **Area Gestore:** Quest'area è dedicata all'ottimizzazione operativa e al supporto per gli amministratori della piattaforma. Fornisce un agente con capacità superiori (un set di tools esteso) capace di eseguire azioni amministrative (es. `unlock_user` o `get_damages`) e analisi complesse (`analyze_vehicle_positioning`). Inoltre comprende un sistema di raccomandazione (basato sul modello Random Forest) che analizza i dati storici per suggerire la gestione ottimale e la redistribuzione dei mezzi nei vari parcheggi.

## 2. Progettazione

### 2.1 Architettura generale

L'architettura di MobiShare (descritta nella relazione PISSIR) è basata su un'applicazione backend monolitica in Node.js (Express) che gestisce le route web, l'autenticazione, la logica principale e la comunicazione MQTT.

Per integrare le funzionalità AI, è stato adottato un approccio simile a quello dell MQTT, ovvero un sistema a servizi esterni (microservizi). È stato creato un server "ponte" (AI-Bridge) in Python (FastAPI) che espone API REST per interagire con il backend Node.js

Questo "AI-Bridge" gestisce i tre servizi principale:

1. **fr\_service (Face Recognition Service)**: Gestisce la registrazione (`/register`) e la verifica (`/verify`) biometrica.
2. **rf\_service (Random Forest Service)**: Gestisce l'addestramento del modello (`/train`) e la generazione di suggerimenti (`/park_suggestions`).
3. **agent\_service (Agent Service)**: Gestisce le query dell'utente verso l'agente (`/query`).

### 2.2 Architettura MVC

L'architettura adotta un pattern **MVC (Model-View-Controller)** per integrare i componenti IA. La logica IA è gestita da un proxy esterno (Fast Api) che viene richiamato dal Controller.

#### Model

è rappresentato dalle collezioni su Firestore, che gestiscono oltre alle entità tradizionali (utenti, corse) sia i nuovi dati specifici per l'AI.

- **Schema Dati**: È stato adottato uno schema ibrido e flessibile per accomodare sia i dati convenzionali sia i *feature vectors* (vettori numerici) richiesti dai modelli di machine learning.
- **Validazione Dati**: La validazione è gestita sia a livello di controller (prima dell'inserimento) sia tramite le security rules di Firestore.
- **Entità AI-Specifiche**: Per supportare le nuove funzionalità, sono state aggiunte le seguenti collezioni :
  - **face\_encodings**: Archivia i dati biometrici (vettori di embedding) necessari per il riconoscimento facciale.
  - **ml\_models/cache**: Salva i metadati, le versioni, le feature utilizzate e le metriche di performance (MAE/RMSE) dei modelli di machine learning addestrati.
  - **agent\_sessions / agent\_memory**: Collezioni usate dal framework Agno per gestire la memoria a breve e lungo termine dell'agente.

## View

L'integrazione AI ha richiesto l'aggiunta di nuovi componenti specifici nell'interfaccia utente:

- **FaceRecognition:** Aggiunta sezione dedicata nel profilo per registrare o eliminare la verifica biometrica e l'aggiunta della sezione di verifica facciale in tutte le schermate di pagamento.
- **Agente:** Una pagina dedicata per comunicare con l'agente chatbot.
- **Suggeritore Gestione:** Dashboard nella sezione parcheggi del gestore, serve a visualizzare i suggerimenti generati dal modello ML sulla distribuzione dei mezzi.

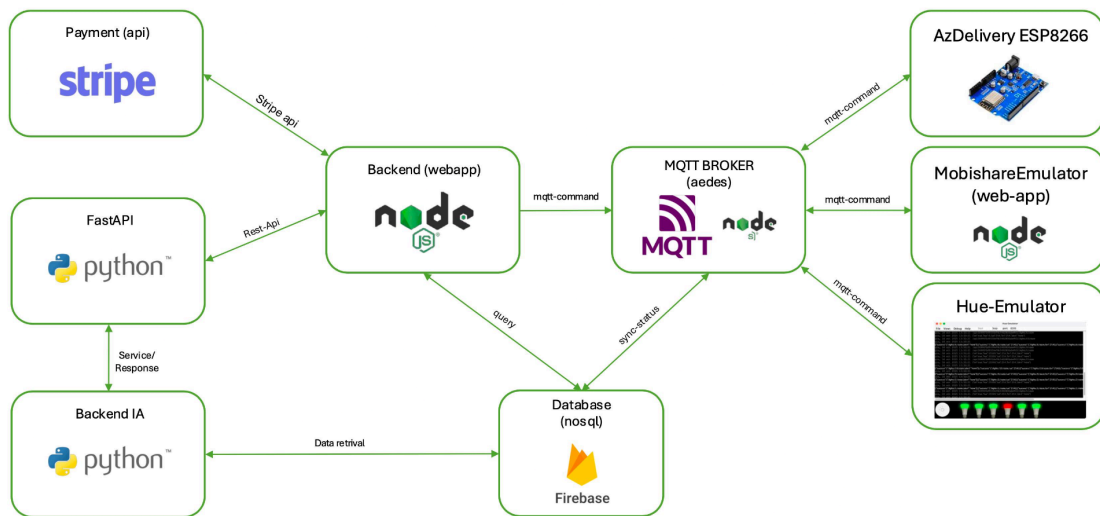
## Controller

La logica usata per l'AI viene gestita da specifiche route API nel server principale che fungono da proxy verso il server Python di FastApi:

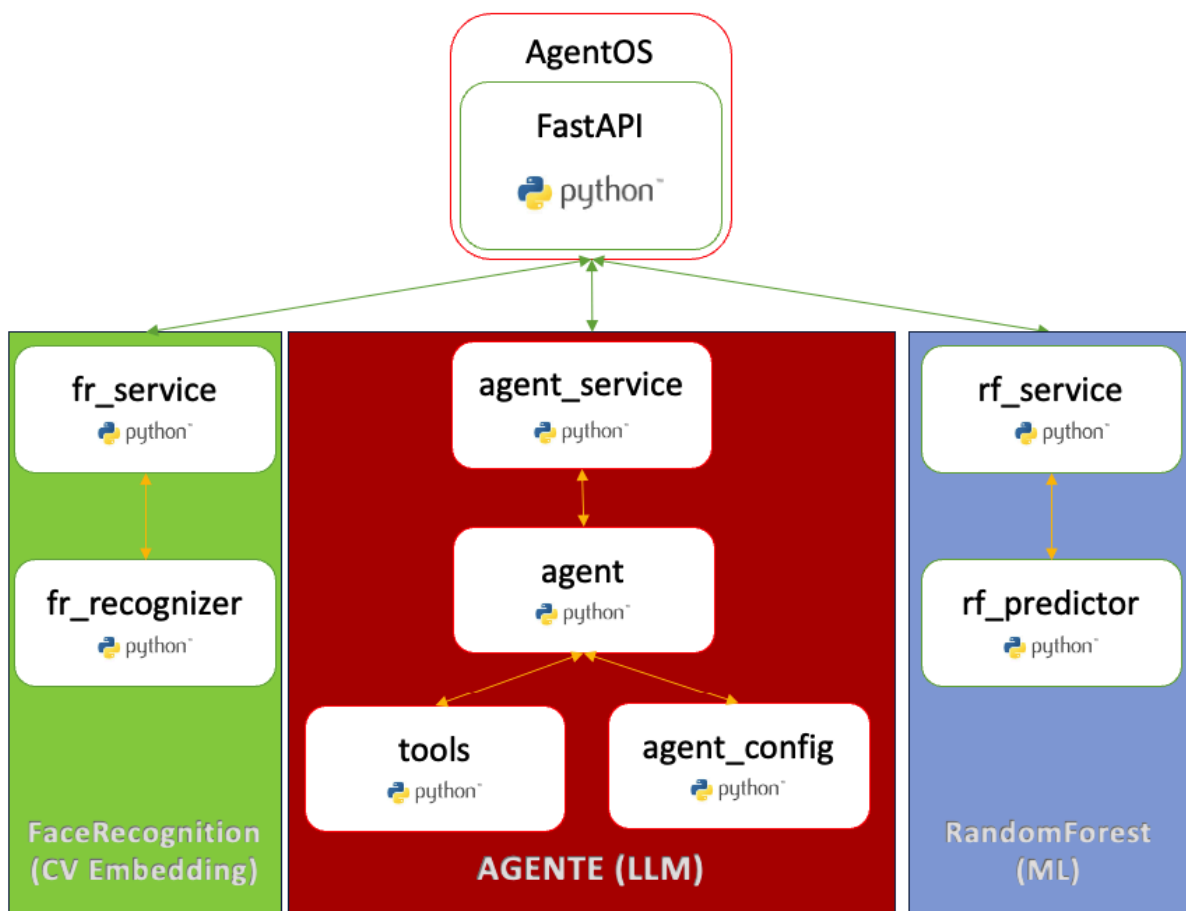
- **Face Recognition:** Il controller espone le route (`/api/face-recognition/...`) che gestiscono il ciclo di vita della biometria. Queste ricevono l'immagine dall'utente (tramite `req.files.image`) verificano l'autenticazione e mediante `axios` inoltrano la richiesta al server FastAPI dove verrà gestita da `fr_service`.
- **Predizioni (ML):** Il controller espone le route (`/api/predizioni/...`) solo ai gestori. Mentre (`/distribuzione-ottimale` e `/addestra-modello`) sono proxy che utilizzano `axios` per inviare la richiesta al server FastAPI dove verrà gestita da `rf_service`.
- **Agente (LLM):** La rotta `/api/chatbot/query` agisce come un proxy per il chatbot, riceve la query dall'utente (aggiungendo dati sull'autenticazione e sulla sessione `user_role` e `user_id`) e la inoltra al server FastAPI dove verrà gestita da `agent_service`

## 2.3 Rappresentazione Architettura

### Generale (con integrazione AI)



### Specifica del Backend AI



## 2.4 Componenti principali

| Componente                 | Descrizione                                                                                                                                                                      |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend Python (FastAPI)   | Gestisce la logica del backend di intelligenza artificiale "AI-BRIDGE". Gestisce le varie chiamate rest provenienti dal front-end e le indirizza all'apposito servizio richiesto |
| Bucket Firestore (Storage) | Storage in cui vengono salvati eventuali dati riguardanti la parte AI (esempio il modello RandomForest)                                                                          |
| Servizio FaceRecognition   | Gestisce la verifica facciale dell'utente per i pagamenti                                                                                                                        |
| Servizio Agent             | Gestisce il chatbox basato sul framework Agno e AgentOS                                                                                                                          |
| Servizio RandomForest      | Gestisce la parte dell'allenamento del Modello "Random Forest" per i suggerimenti sulla gestione dei parcheggi                                                                   |

## 2.5 Struttura del database (Firestore)

### Motivazioni della scelta di un database NoSQL (Specialmente Firestore)

- Firestore ci fornisce una soluzione già deployata, utilizzabile contemporaneamente da tutti i membri del gruppo e dai vari server in esecuzione.
- Ci offre grande flessibilità nella struttura dei dati, caratteristiche che abbiamo ritenuto fondamentali per integrare il modulo di Applicazioni Intelligenti in particolare modo:
  - Per gli algoritmi di machine learning come la RandomForest (per i suggerimenti sulla distribuzione dei mezzi) che ha bisogno di avere sempre dati aggiornati e può essere usato come cache per salvare i modelli (Da usare poi su vari dispositivi)
  - FaceRecognition: necessità di poter verificare un utente da qualunque dispositivo.
  - L'agent: in quanto necessita di avere dati sempre aggiornati per poter fornire risposte sempre attuali (in termini di dati) e per permettergli di salvare le sessioni/memorie.

### Struttura dei Documenti

Per supportare le nuove funzionalità, la struttura del database Firestore è stata estesa:

Documento: face\_encoding

|           |         |                                          |
|-----------|---------|------------------------------------------|
| createdAt | Boolean | Stato registrazione verifica facciale    |
| encoding  | Array[] | Contiene l'encoding del viso dell'utente |
| source    | String  |                                          |
| user      | String  | Riferimento al documento dell'utente     |

Documento: User

|                      |         |                                       |
|----------------------|---------|---------------------------------------|
| faceRegistered       | Boolean | Stato registrazione verifica facciale |
| faceRegistrationDate | Date    | Data Registrazione verifica facciale  |

Documento: ml\_models

|                 |         |                                               |
|-----------------|---------|-----------------------------------------------|
| feature_columns | Array[] | Contiene la lista delle "feature" del modello |
| last_training   | Date    | Ultima data di training del modello           |
| mode_path       | String  | Path di firebase dove è salvato il modello    |
| performance     | Array[] | Contiene le metriche di performance MAE-RMSE  |

Documento: ml\_cache

|                |                                                        |                                                    |
|----------------|--------------------------------------------------------|----------------------------------------------------|
| generated_at   | Timestamp                                              | Data di generazione dei suggerimenti               |
| metadata       | Map (suggestion_percentage, total_prediction_analyzed) | Contiene dei dati "secondari"                      |
| plot           | Map(bar, heatmap, line)                                | Path per i plot generati                           |
| plot_generated | Boolean                                                | Flag per controllare che i plot siano stati creati |
| suggestion     | Array[]                                                | Lista dei suggerimenti per i parcheggi             |

Documento: agent\_memory

|            |           |                                       |
|------------|-----------|---------------------------------------|
| agent_id   | String    | id dell'agente                        |
| input      | String    | Input dell'agente                     |
| memory     | String    | Contenuto della memoria               |
| memory_id  | String    | Riferimento alla memoria per l'agente |
| topic      | Array[]   | Argomenti                             |
| updated_at | Timestamp | Orari di aggiornamento                |
| user_id    | String    | Id dell'utente                        |

|                      |           |                                                                                                                                         |
|----------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------|
| agent_data           | String    | "{"agent_id": "738901a7-5e87-415b-8cf4-665455fc3fc5", "model": {"provider": "Google", "name": "Gemini", "id": "gemini-2.0-flash-001"}}" |
| agent_id             | String    | Id Agente                                                                                                                               |
| conversation_history | Array[]   | Query: "user input", Response: "agent response"                                                                                         |
| created_at           | Timestamp | Timestamp creazione sessione                                                                                                            |
| last_message         | String    | Ultimo messaggio                                                                                                                        |
| runs                 | String    | Contenuto di esecuzione (regole+input+identità)                                                                                         |
| session_data         | String    | Id dell'utente                                                                                                                          |
| session_id           | String    | Uguale a Agent_id                                                                                                                       |
| session_type         | String    | "agent"                                                                                                                                 |
| summary              | null      |                                                                                                                                         |
| updated_at           | Timestamp | Timestamp di aggiornamento                                                                                                              |
| user_id              | String    | UID utente                                                                                                                              |
| user_role            | String    | "gestore" o "utente"                                                                                                                    |

### 3 Funzionalità AI

Nonostante il corso sia più incentrato sulla parte LLM abbiamo voluto comunque implementare anche qualcosina riguardante gli altri argomenti, così da poter creare un progetto il più esteso possibile con la maggior varietà di argomenti.

#### 3.1 FaceRecognition (Embedding e augmentation)

Per la parte di *"ComputerVision"* del progetto abbiamo deciso di implementare un sistema di riconoscimento facciale per la verifica a due fattori nei pagamenti (come alternativa alla password), per far ciò abbiamo utilizzato la libreria **face\_recognition**. Questa libreria fornisce un'implementazione pre-addestrata di un modello per il riconoscimento dei volti. (basato su dlib e deep learning per l'estrazione dei face embeddings. Questi embedding rappresentano il volto come un vettore numerico, permettendo il confronto tra volti diversi tramite metriche di distanza (es. L2 o cosine similarity), senza dover addestrare manualmente una rete neurale convoluzionale. source: gemini). Successivamente, per migliorare il riconoscimento in condizioni reali (luci, angolazioni della testa, fotocamere diverse (alcune fanno il mirroring)) è stata fatta un'ulteriore procedura di augmentation, che consiste nell'applicare modifiche all'immagine originaria (fornendo più campioni su cui fare l'encoding). Le augmentation che abbiamo implementato sono:

- **Flip orizzontale:** Per gestire fotocamere che scattano immagini specchiate.
- **Rotazioni leggere (+/-10 gradi):** per simulare le diverse inclinazioni del volto.
- **Conversione in scala di grigi:** per ridurre la dipendenza e le influenze delle condizioni della luce.

Il processo del recognizer è suddiviso in due parti::

1. **Registrazione (save\_encoding):**

- L'utente invia un'immagine.
- Il sistema esegue una Augmentation per poter generare più dati e aumentare l'efficacia.
- Gli embedding di tutte le immagini aumentate vengono estratti e salvati nella collezione `face_encoding` (ogni encoding è associato a un utente).

2. **Verifica (verify\_face):**

- L'utente invia una nuova immagine per la verifica.
- Il sistema estrae l'embedding di questa nuova immagine.
- Confronta l'embedding con l'encoding di quell'utente.
- La verifica ha successo se almeno uno degli encoding salvati corrisponde (con una tolleranza di 0.5).

## 3.2 RandomForest (ML-Supervisionato)

Per la parte relativa al modello di ML, è stato scelto di implementare un modello di supervisionato, in quanto i dati sono già ben strutturati. Il modello che abbiamo scelto è "Random Forest", che è appunto un algoritmo di apprendimento supervisionato basato su alberi decisionali. Per la parte del modello abbiamo deciso di implementare un modello di predizione Supervisionato (in quanto i dati provengono da colonne ben distinte).

Per poter addestrare in modo efficace il modello abbiamo realizzato uno script di Node con lo scopo di popolare il documento `rides` con un numero definito di corse, queste simulano il funzionamento di mobishare e gestiscono delle casistiche reali... sono presenti due versioni del sistema di ML, una che funziona interamente con firestore e un'altra interamente offline (con un database locale json), quest'ultimo è stato usato per testare il generatore con vari volumi di dati (per non eccedere il limite di operazioni di lettura e scrittura offerti da firebase).

*NB: la versione caricata su firestore ha comunque una dimensione limitata (circa 2100) in quanto con grossi valori si raggiungeva subito il limite di letture e scritture (esempio se si effettuano alcuni aggiornamenti del modello durante la giornata.. questo numero permette di fare circa 15 aggiornamenti massimi giornalieri).*

Il processo del predictor è il seguente:

1. **Preparazione** (`prepare_training_data`):
  - Il sistema legge i dati dalla collezione `rides`.
  - Estrarre le feature significative: `['parking_id', 'hour', 'fascia_oraria', 'day_of_week', 'month', 'is_weekend', 'previous_day_demand', 'previous_week_demand', 'mezzo_type_encoded']`
2. **Addestramento** (`train_model`):
  - I dati vengono suddivisi in set di training e test (`train_test_split: 0.2 (80/20)`).
  - Viene addestrato il modello `RandomForestRegressor` (da `sklearn`).
  - Il modello addestrato (`.pkl`) viene poi salvato su Firebase Storage.
3. **Valutazione**
  - L'efficacia del modello è valutata tramite le metriche indicate nel paragrafo Analisi Prestazioni.
4. **Suggerimenti**:
  - Il servizio `/park_suggestions` (in `rf_service.py`) utilizza il modello addestrato per predire la domanda futura e confrontarla con i mezzi attualmente disponibili in quel parcheggio.
  - Questo genera suggerimenti per il gestore (es. "Sposta 2 mezzi da Parcheggio A a Parcheggio B").

## Analisi Prestazioni

Per l'analisi del modello abbiamo usato le metriche **MAE** e **RMSE**, queste metriche ci permettono di valutare l'efficacia misurandone l'errore medio tra i valori previsti e quelli reali.

**MAE (Mean Absolute Error):** L'Errore Medio Assoluto, che misura l'errore medio delle previsioni (nel nostro caso, il numero medio di mezzi che la previsione sbaglia).

**RMSE (Root Mean Square Error):** La Radice dell'Errore Quadratico Medio, che è simile al MAE ma penalizza maggiormente gli errori più grandi (previsioni molto errate).

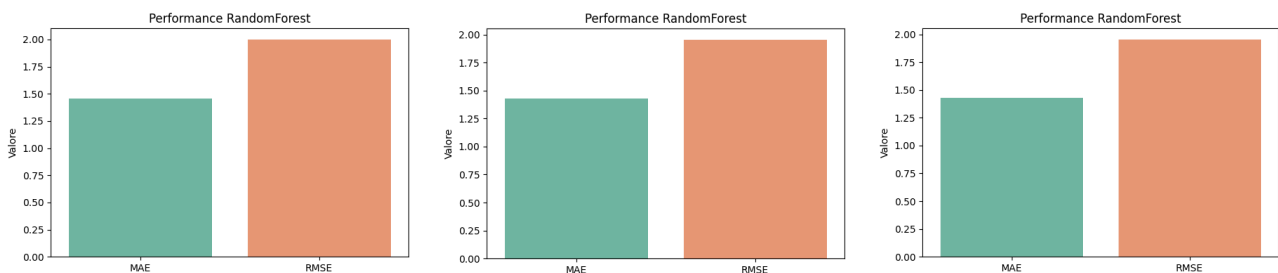
## Fase 1: Dati Limitati (10 Giorni)

Inizialmente, abbiamo generato un set di dati pari a 10 giorni in quanto appunto non volevamo esagerare... abbiamo ottenuto un totale di circa 2.000 corse totali che con l'aggregazione sono diventate 571 righe.

```
Statistiche finali:
- Totale corse: 2000
- Parcheggi: 4
- Mezzi: 16
- Parcheggio Asia: 2 mezzi, 526 corse partite
- Parcheggio Giulia: 3 mezzi, 504 corse partite
- Parcheggio Magenta: 10 mezzi, 479 corse partite
- Parcheggio Matteo: 1 mezzi, 491 corse partite
```

- **Test 1:** Con 50 stimatori e una profondità (depth) di 9, i risultati sono stati abbastanza scarsi, abbiamo ottenuto come Performance - **MAE 1.46** e **RMSE 2.00**. che equivale a un errore medio tra 1 e 2 mezzi all'ora (troppo)
- **Test 2 e 3 (Tuning Parametri):** Abbiamo provato a modificare un pò i parametri del modello, aumentando rispettivamente gli stimatori a 100 e poi a 150 ma tutto ciò ci ha portato a ottenere solo dei miglioramenti marginali Performance - **MAE 1.43**, **RMSE 1.95**.

**Resoconto 1:** L'ottimizzazione dei parametri su un set di dati piccolo si è rivelata poco efficace in quanto il modello non aveva abbastanza campioni per apprendere i pattern.



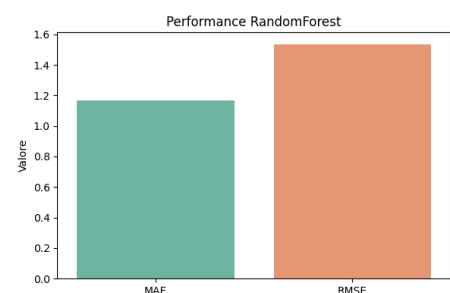
Test 1 (MAE 1.46 e RMSE 2.00)    Test 2(MAE: 1.44 RMSE: 1.98)    Test3 (MAE: 1.43 RMSE: 1.95)

## Fase 2: Aumento dei Dati (90 Giorni)

Basandoci sui risultati precedenti abbiamo appurato che fosse necessario aumentare di un bel pò il numero di dati su cui allenare il modello in quanto i precedenti erano insufficienti

```
- Totale corse: 18163
- Parcheggi: 4
- Mezzi: 16
- Parcheggio Asia: 4 mezzi, 4509 corse partite
- Parcheggio Giulia: 7 mezzi, 4552 corse partite
- Parcheggio Magenta: 3 mezzi, 4582 corse partite
- Parcheggio Matteo: 2 mezzi, 4520 corse partite
```

- **Test 4:** Abbiamo generato un nuovo set di dati di 90 giorni (circa 18.163 corse). Mantenendo gli stessi parametri (150 stimatori, depth 9) e abbiamo notato che le performance sono migliorate significativamente: **MAE 1.17** e **RMSE 1.54**.



**Resoconto 2:** Questo test ha confermato che il numero dei dati era la causa principale delle scarse performance del modello

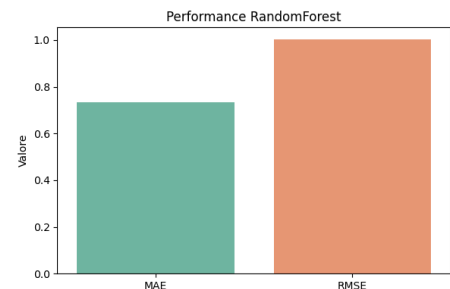
### Fase 3: Aumento delle Feature e Dati a 6 Mesi

Sebbene migliorato, il modello aggrega tutte le corse per parcheggio ignorando appunto il tipo di mezzo. Perciò abbiamo deciso di rendere il modello più specifico e meno generico

```
Totale corse: 41061
Parcheggi: 4
Mezzi: 16
Parcheggio Asia: 3 mezzi, 10209 corse partite
Parcheggio Giulia: 4 mezzi, 10275 corse partite
Parcheggio Magenta: 4 mezzi, 10315 corse partite
Parcheggio Matteo: 5 mezzi, 10262 corse partite
```

**Test 5:** In questo test abbiamo fatto 2 cose:

- **Aggiunta Feature:** Abbiamo incluso `mezzo_type_encoded` (Bicicletta, Bicicletta Elettrica e Monopattino) permettendo al modello di apprendere pattern più specifici e meno generici
- **Aumento ulteriore dei dati:** Abbiamo esteso il dataset a **6 mesi**



**Resoconto 3:** Abbiamo ottenuto un **MAE di 0.73** e un **RMSE di 1.00**. È un buon miglioramento rispetto all'inizio e per questo (vista anche la randomicità e la non troppa "certezza" dei dati generati randomicamente dal nostro script) abbiamo deciso di mantenere buono questo modello

## 3.3 Agente

Mobishare include un Agente specializzato che serve a migliorare il supporto agli utenti (clienti) e ottimizzare l'operatività dei gestori (chiedendo analisi o informazioni sullo stato della piattaforma).

### Architettura e tecnologie usate

- **Framework:** L'agente è sviluppato con il framework **Agno**.
- **Integrazione:** Il sistema è servito tramite **AgentOS** che si integra direttamente con FastAPI (esponendo l'interfaccia sulla porta 7777 che rende possibile la visualizzazione di una dashboard da: [os.agno.com](https://os.agno.com) )
- **Modello LLM:** Come modello linguistico e di reasoning è stato scelto **Gemini 2.5 Flash**.
- **Gestione della Memoria:** L'architettura è ideata per essere stateful, ovvero mantiene la memoria della conversazione per contestualizzare le richieste.
- **Controllo Accessi:** Il sistema è role-based, si basa sul ruolo dell'utente che effettua la richiesta.
- **Dati:** L'agente opera direttamente sui dati in tempo reale da Firestore, garantendo che le risposte siano sempre aggiornate in base allo stato attuale della piattaforma.

## Struttura Agente

L'agente Agno viene istanziato mediante la funzione `get_mobishare_agent()` con i seguenti parametri:

- **Model (Modello):** Specifica quale LLM usare. In questo caso, è impostato su Gemini 2.5 Flash. Questosi occupa sia della comprensione del linguaggio che del reasoning.
- **Instructions (Istruzioni):** Definisce l'identità, il tono e le regole di comportamento dell'agente. Queste vengono definite in `AGENT_IDENTITY` e le regole di accesso basate sul ruolo (`user_role`) fornito al momento della creazione.
- **Tools (Strumenti):** Definisce gli strumenti pratici che può usare l'agente, nel nostro caso è stato collegando la classe `ChatbotTools` che contiene tutte le funzioni Python che l'agente può decidere di eseguire.
- **Memory (Memoria):** L'agente è stateful e utilizza un sistema di memoria a due livelli:
  - **Memoria a Breve Termine:** Abilitata da `add_history_to_context=True`, permette all'agente di ricordare le domande e le risposte precedenti all'interno della stessa sessione.
  - **Memoria a Lungo Termine (Agentic):** `enable_agentic_memory=True` e `add_memories_to_context=True` Consente all'agente di memorizzare e recuperare gli elementi chiave sull'utente tra sessioni diverse (es. "l'utente si chiama mirko e gli piacciono i monopattini").
- **User\_ID:** Serve a "contestualizzare" la memoria, i tools etc etc assicurando che l'agente operi solo sui dati pertinenti a quell'utente.
- **DB:** collega l'agente al database Firestore.
- **Markdown:** `markdown=True` indica all'agente di formattare le sue risposte in Markdown.

## Tools

L'agente per rispondere a determinate richieste può avvalersi dei Tools. I tools sono delle funzioni Python (definite da noi) che l'agente può decidere di usare per recuperare dati dal database e per eseguire azioni in base alle richieste dell'utente e anche in base al ruolo. (esempio, un utente non può chiedere informazioni su altri utenti e non può chiedere o trattare argomenti fuori dal suo "contesto" di utente (esempio lo stato dei parcheggi/manutenzione etc etc). Per implementarli abbiamo usato la classe Toolkit di Agno.

Nel progetto, i tools sono implementati come metodi all'interno della classe `ChatbotTools`

- **Istanza:** La classe `ChatbotTools` viene inizializzata con `user_id` e `user_role` dell'utente

- **Scoping:** I tools destinati agli utenti (es. `get_user_rides`, `get_payment_history`) utilizzano `self.user_id` per filtrare le query al database. Questo garantisce che un utente possa *solo* vedere le proprie corse o i propri pagamenti.
- **Controllo dei Permessi:** I tools amministrativi (es. `get_suspended_users`, `unlock_user`, `get_damages`) contengono un controllo esplicito. All'inizio della funzione, verificano `if self.user_role != 'gestore'`, restituendo un messaggio di accesso negato se un utente standard tenta di invocarli.

### Funzionalità (Tools) per l'Utente

|                                                              |                                                                                                                                      |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Storico Corse<br><code>get_user_rides</code>                 | Recupera le corse effettuate dall'utente, con possibilità di applicare filtri temporali                                              |
| Storico Pagamenti<br><code>get_payment_history</code>        | Fornisce la liste delle ricariche effettuate                                                                                         |
| Saldo Utente                                                 | Restituisce il saldo in euro, i punti fedeltà e verifica la presenza di eventuali corse in stato in_riepilogo (pagamenti in sospeso) |
| Analisi Pattern<br><code>analyze_ride_patterns</code>        | Analizza i dati storici delle corse per suggerire orari, mezzi preferiti e consumo di credito medio                                  |
| Riscatto Punti<br><code>redeem_shop_package</code>           | Permette il riscatto di pacchetti fedeltà, scalando i propri punti e incrementando il saldo in euro                                  |
| Visualizza pacchetti punti<br><code>get_shop_packages</code> | Fornisce la lista di tutti i pacchetti punti disponibili                                                                             |
| Mezzi disponibili<br><code>get_available_vehicles</code>     | Trova i veicoli con stato disponibile                                                                                                |

### Funzionalità (Tools) per il gestore

|                                                                          |                                                                                                                           |
|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Visualizza Utenti Sospesi<br><code>get_suspended_users</code>            | Recupera la lista degli utenti con stato profilo sospeso                                                                  |
| Gestione Utenti Sospesi<br><code>unlock_user</code>                      | Recupera e cambia stato da sospeso a attivo l'account dell'utente tramite una ricerca con nome o email o id dell'utente   |
| Ricerca di utenti<br><code>find_users_by_details</code>                  | Permette la ricerca degli utenti per email o nome                                                                         |
| Storico Danni<br><code>get_damages</code>                                | Recupera la lista dei mezzi in manutenzione                                                                               |
| Analisi Posizionamento Mezzi<br><code>analyze_vehicle_positioning</code> | Analizza i dati storici delle corse negli ultimi 7 giorni, fornendo informazioni e suggerimenti per la gestione dei mezzi |

|                                                            |                                                                    |
|------------------------------------------------------------|--------------------------------------------------------------------|
| Sentiment Analysis<br><code>_get_sentiment_from_llm</code> | Analizza i “sentimenti” dei feedback (positivo, negativo o neutro) |
|------------------------------------------------------------|--------------------------------------------------------------------|

## Definizione Identità

La definizione dell'identità dell'agente è una componente chiave per garantire un'interazione coerente, sicura e allineata agli obiettivi del nostro sito. L'identità dell'agente di Mobishare è stata definita tramite un prompt di sistema (AGENT IDENTITY) che ne stabilisce i confini operativi, il tono e le regole di comportamento.

- **Tono e Personalità:** L'agente si presenta come un assistente AI specializzato per MobiShare. Il suo tono è impostato per essere professionale, amichevole e focalizzato sui dati (specificando anche chi non è).
- **Dominio di Competenza:** Il suo compito è assistere clienti e personale interno esclusivamente sui servizi di mobilità condivisa offerti. L'agente è strettamente focalizzato su Monopattini Elettrici, Biciclette e Biciclette a pedalata assistita (eBike), ed è stato istruito a non menzionare auto o altri mezzi di trasporto.
- **Gestione della Memoria e Contesto:** L'agente è istruito a mantenere il contesto della conversazione, ricordando il nome e i dettagli dell'utente e facendo riferimento alle interazioni precedenti. Segue inoltre regole stringenti sulla privacy, come non memorizzare informazioni sensibili (dati di pagamento, dati personali) e aggiornare la memoria solo con informazioni nuove e significative.
- **Regole di Interazione:** Come definito nel `RESPONSE_SYSTEM_PROMPT`, l'agente deve rispondere in italiano in modo conciso ma completo, usando i dati recuperati dal database e l'ID utente per fornire risposte adeguate. È istruito a **non menzionare mai dati interni di processo** all'utente, come `reasoning_steps` o `user_role`.
- **Adattamento al Ruolo:** L'identità dell'agente si adatta dinamicamente in base al ruolo dell'utente (Utente o Gestore). Questo limita i tools disponibili all'utente e assicura che l'agente rispetti i confini di accesso alle informazioni.

## Processo di Reasoning

Prima di eseguire qualsiasi azione l'agente analizza la richiesta dell'utente utilizzando un prompt dedicato `REASONING_SYSTEM_PROMPT`, questo suddivide il problema in varie fasi:

**Analisi Richiesta:** Determinare l'intento dell'utente (`assistenza`, `pagamenti`, ...).

- **Identificare i Dati Necessari:** Capire quali informazioni mancano per soddisfare la richiesta e quali dati devono essere recuperati.
- **Pianificare le Azioni:** Stabilisce le operazioni da fare e quali tools sono necessari per recuperare i dati (utilizzando il `user_id` e il `db` a sua disposizione).
- **Rispettare i Ruoli:** Eseguire l'intera analisi nel rispetto dell'identità (`user_id`) e del ruolo (Utente o Gestore) dell'utente (`user_role`).